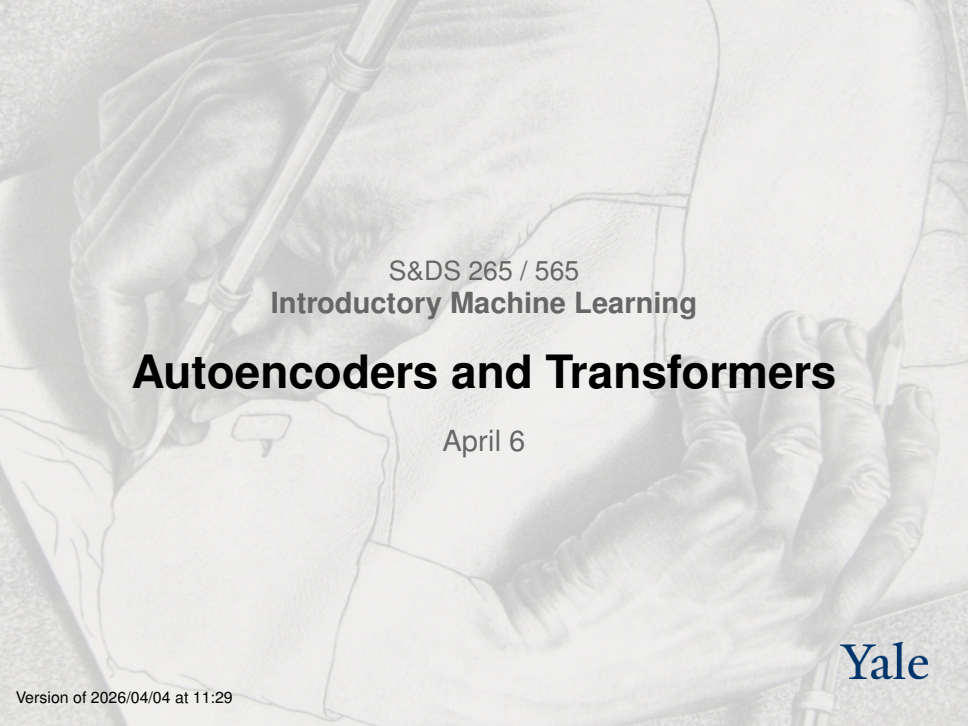




S&DS 265 / 565
Introductory Machine Learning

Autoencoders and Transformers

April 6

Yale

Reminders

- Assignment 4 due Thursday
- Assignment 5 (last!) posted Thursday
- Final exam, May 4 at 2pm
- Review sessions TBA

Quiz 4

Quiz Summary

Section Filter ▾

 Student Analysis

 Item Analysis

 Average Score

90%

 High Score

100%

 Low Score

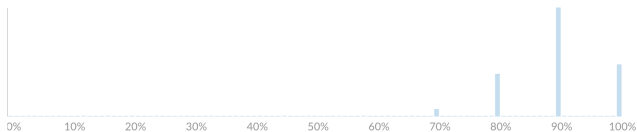
70%

 Standard
Deviation

0.76

 Average Time

18:57



Quiz 4

How does Deep Q-Learning use deep neural networks?

Discrimination
Index (?)

37% answered
correctly

A neural network with states as inputs is used to define a Q-function	76 respondents	59 %	
A neural network is used to extract features of states	5 respondents	4 %	
A neural network is used to define a value function		0 %	
All of the above	48 respondents	37 %	✓
None of the above		0 %	

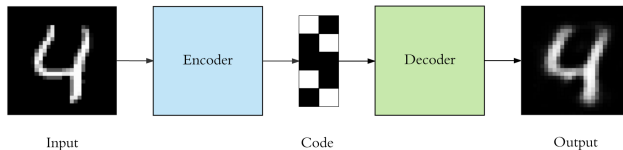
For today

- Autoencoders
- Transformers

Autoencoders

- Unsupervised learning methods
- Squeeze high dimensional data through a “bottleneck” of lower dimension
- Train to minimize reconstruction error

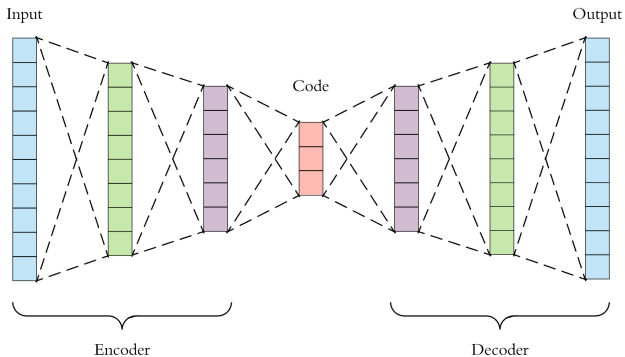
Autoencoders



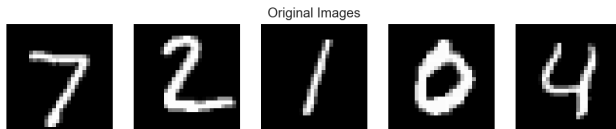
Important aspects

- Unsupervised: No labels used, discovers useful features of input
- Compression: Code reduces dimension of data
- Lossy: Input won't be reconstructed exactly
- Trained: The compression algorithm is learned for specific data

Deep architecture



MNIST data



$$28 \times 28 = 784 = D$$

It cuts both ways...



numpy



keras

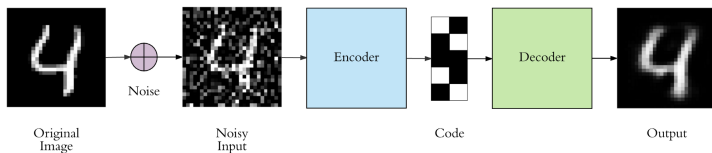
Implementation using Keras

```
1  input_size = 784
2  hidden_size = 128
3  code_size = 32
4
5  input_img = Input(shape=(input_size,))
6  hidden_1 = Dense(hidden_size, activation='relu')(input_img)
7  code = Dense(code_size, activation='relu')(hidden_1)
8  hidden_2 = Dense(hidden_size, activation='relu')(code)
9  output_img = Dense(input_size, activation='sigmoid')(hidden_2)
10
11 autoencoder = Model(input_img, output_img)
12 autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
13 autoencoder.fit(x_train, x_train, epochs=5)
```

Adam optimizer

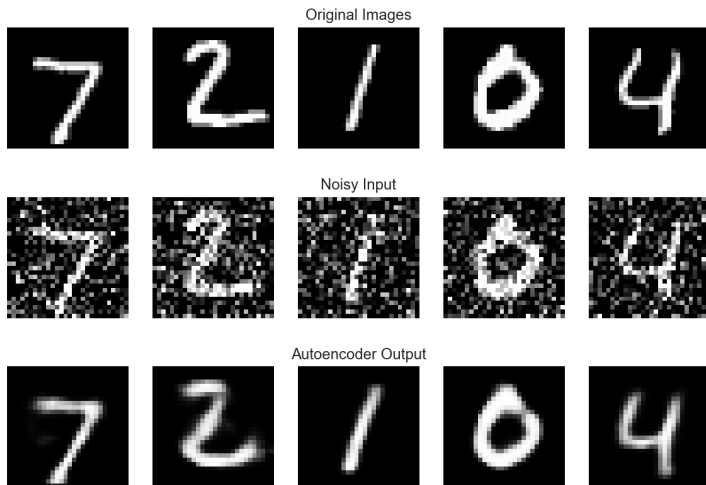
- Variant of stochastic gradient descent where separate learning rate (step size) is maintained for each network weight (parameter)
- Each step size adapted as learning progresses based on moments of the derivatives

Variant: Denoising autoencoder

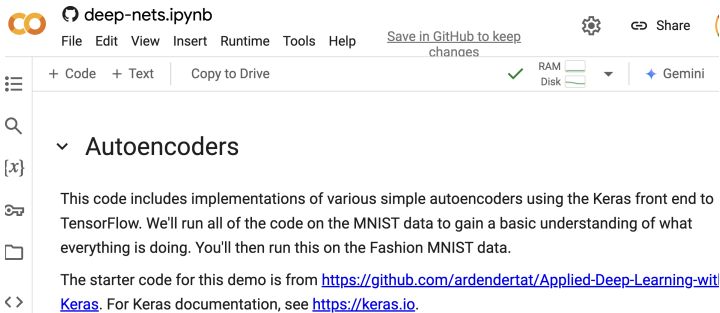


- Feed in noisy data
- Train to match to denoised data

Example result on MNIST



Demo



The screenshot shows a JupyterLab interface. At the top, the notebook title is 'deep-nets.ipynb'. Below the title is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. To the right of the menu bar is a 'Save in GitHub to keep changes' link, a settings gear icon, and a 'Share' button. Below the menu bar is a toolbar with '+ Code', '+ Text', 'Copy to Drive', a green checkmark, 'RAM' and 'Disk' usage indicators, and a 'Gemini' button. On the left side, there is a sidebar with icons for a menu, search, a variable '[x]', a key, a folder, and a double arrow. The main area displays the notebook content, which starts with a section header 'Autoencoders' and a paragraph of text.

deep-nets.ipynb

File Edit View Insert Runtime Tools Help

Save in GitHub to keep changes

+ Code + Text Copy to Drive

RAM Disk

Gemini

Autoencoders

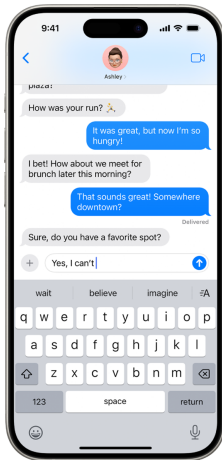
This code includes implementations of various simple autoencoders using the Keras front end to TensorFlow. We'll run all of the code on the MNIST data to gain a basic understanding of what everything is doing. You'll then run this on the Fashion MNIST data.

The starter code for this demo is from <https://github.com/ardendertat/Applied-Deep-Learning-with-Keras>. For Keras documentation, see <https://keras.io>.

Summary: Autoencoders

- Autoencoders compress the input and then reconstruct it
- Bottleneck forces extraction of useful features
- Will overfit and “memorize” the data
- Overfitting mitigated by denoising autoencoders

Large Language Models



<https://support.apple.com/>

We've all used language models many times today

Home assistants

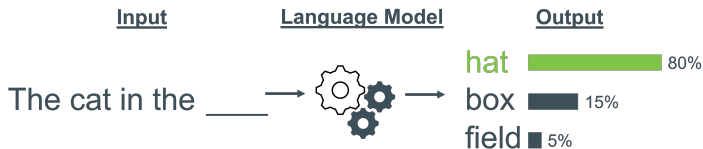


Many uses of language models

- Speech recognition (Siri, Alexa, etc.)
- Machine translation (Google translate)
- Text compression
- Texting
- Email completion
- Image captioning
- ...

Text generation

- Words generated one-by-one
- A word is chosen by sampling from a probability distribution
- Result is purely synthetic text—generative AI



Language models

- Because it predicts the next word, a language model gives a way of *generating* text (through sampling)

$$\begin{aligned} P(\text{"the whole forest had been anesthetized"}) = & \\ & P(\text{"the"}) \times P(\text{"whole"} \mid \text{"the"}) \\ & \times P(\text{"forest"} \mid \text{"the whole"}) \\ & \times P(\text{"had"} \mid \text{"the whole forest"}) \\ & \times P(\text{"been"} \mid \text{"the whole forest had"}) \\ & \times P(\text{"anesthetized"} \mid \text{"the whole forest had been"}) \end{aligned}$$

Modern language models

The algorithm assigns a “score” to possible next words w :

$$\text{score}(\underset{\substack{\uparrow \\ \text{possible next word}}}{w}; \overbrace{w_1, \dots, w_n}^{\text{previous words}})$$

- score is computed as the similarity between two high-dimensional vectors—we’ll explain how!
- it is then converted into a probability

Key steps

- *Tokenize* the text into a fixed vocabulary
- *Embed* the tokens into high-dimensional vectors
- *Mix* the embeddings in the context using “Attention”
- *Map* the resulting vectors using a neural network

Tokenization

The vocabulary problem

- Word embeddings we saw constructed for 400,000 word vocabulary
- We each know $\approx 40\text{K}$ words
- What if a word is not in the vocabulary?
- Need to be able to process—and generate—arbitrary text

Tokenization

Fugetsu-Do, a mochi store in the Little Tokyo neighborhood of Los Angeles, has existed for 121 years, with a lasting impact on its community.

Billy was a very baaaaaaaaad goat.

This is another \$%^*ish example.

This is what we see

Tokenization

Fugetsu-Do, a mochi store in the Little Tokyo neighborhood of Los Angeles, has existed for 121 years, with a lasting impact on its community.

Billy was a very baaaaaaaad goat.

This is another \$%^*ish example.

Text

Token IDs

This is how the tokenizer groups bytes into tokens

Tokenization

```
[77845, 19209, 84, 12, 5519, 11, 264, 4647, 14946, 3637, 304, 279, 15013, 27286, 12818, 315, 9853, 12167, 11, 706, 25281, 369, 220, 7994, 1667, 11, 449, 264, 29869, 5536, 389, 1202, 4029, 382, 97003, 574, 264, 1633, 293, 29558, 33746, 329, 54392, 382, 2028, 374, 2500, 400, 46999, 9, 819, 3187, 382]
```

Text

Token IDs

This is what the LLM sees—the “atoms” of an LLM

Tokenizer properties

- *Reversible*: Can convert from tokens back to original text
- *General*: Works on arbitrary text, even very different from training
- *Compressive*: Each token is about 4 bytes
- *Statistical*: Will break strings into frequently occurring pieces

Byte level handles any language

Fugetsu-Do, a mochi store in the Little Tokyo neighborhood of Los Angeles, has existed for 121 years, with a lasting impact on its community.

ロサンゼルスのリトル東京地区にある餅屋「風月堂」は 121 年間存在し、地域社会に永続的な影響を与え続けています。

This is what we see

Byte level handles any language

Fugetsu-Do, a mochi store in the Little Tokyo neighborhood of Los Angeles, has existed for 121 years, with a lasting impact on its community.

ロサンゼルスのリトル東京地区にある「00000」は 121 00間存在し、地00社会に00000的な影000を与え000けています。

Text

Token IDs

This is how the tokenizer groups bytes into tokens

Byte level handles any language

```
[77845, 19209, 84, 12, 5519, 11, 264, 4647, 14946, 3637, 304, 279, 15013, 27286, 12818, 315, 9853, 12167, 11, 706, 25281, 369, 220, 7994, 1667, 11, 449, 264, 29869, 5536, 389, 1202, 4029, 382, 42634, 60868, 16073, 3484, 120, 33710, 22398, 16144, 37823, 20251, 33710, 14276, 109, 47653, 30590, 24775, 20230, 30591, 30369, 165, 97, 227, 23602, 233, 13177, 19817, 101, 9953, 45144, 224, 10646, 15682, 220, 7994, 75677, 112, 56965, 48706, 15024, 5486, 30590, 35722, 253, 61337, 38093, 20230, 30320, 116, 163, 90183, 9554, 26854, 58322, 165, 253, 123, 30512, 58318, 58942, 163, 90183, 76622, 38144, 61689, 3490]
```

Text

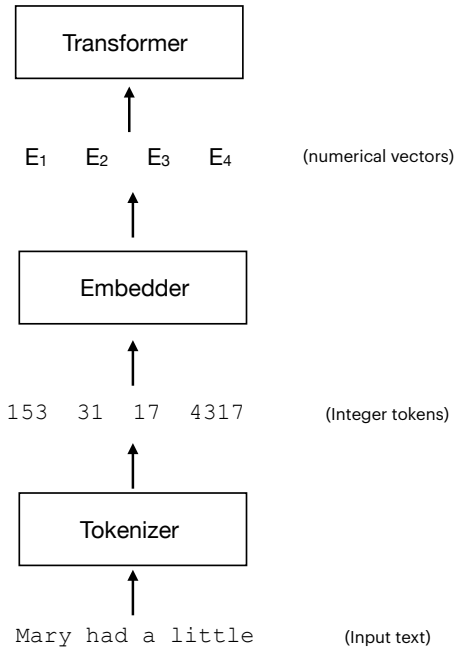
Token IDs

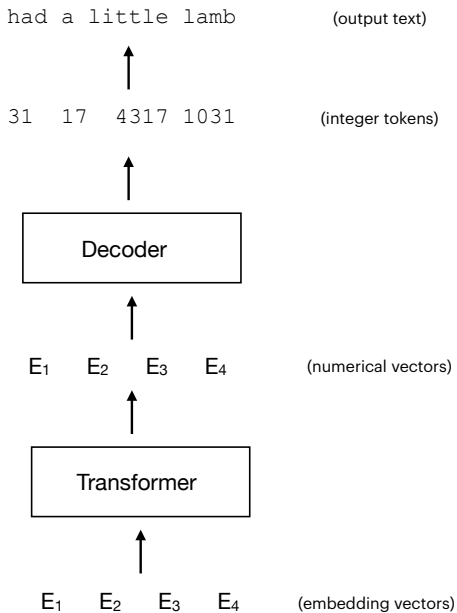
This is what the LLM sees—the “atoms” of an LLM

Tokenizer issues

Tokenization is a messy topic—can lead to troublesome artifacts in LLMs:

- Very general since computed from raw byte streams (Unicode)
- Uneven treatment of different languages
- Failure to recognize equivalence of different uses of same word
- Poor handling of numbers





Word embeddings (review)

Puzzle

- Words are “symbols” — banana, apple, Microsoft
- How do we represent words in a form that a neural network can process?

Nietzsche: “Words are but symbols for the relations of things to one another and to us; nowhere do they touch upon absolute truth.”

Intuition

- Represent words by vectors to capture similarity in usage
- Similar words will appear with similar words (self-referential)
- Words are characterized by the words they co-occur with
- Purely statistical

Constructing embeddings

`word2vec:`

- Based on a very simple language model

Constructing embeddings

word2vec:

- Based on a very simple language model
- Predict *surrounding* words from current word, rather than the next word

Constructing embeddings

word2vec:

- Based on a very simple language model
- Predict *surrounding* words from current word, rather than the next word
- A model of *nearby* words $p_{\text{near}}(w_2 \mid w_1)$

Constructing embeddings

Language model is formed by

$$p_{\text{near}}(w_2 | w_1) \propto \exp \left(\varphi(w_2)^T \varphi(w_1) \right)$$

where the vectors $\varphi(w)$ are the parameters of the model

- Embeddings are “tuned” to predict nearby words

Mikolov et al. at Google (2014) “Distributed representations of words.”

Here $a^T b$ means the “inner product” of the vectors: $a^T b = a_1 \cdot b_1 + a_2 \cdot b_2 + \dots + a_d \cdot b_d$. Two vectors are similar if their inner product is large. It’s related to the cosine of the angle between them in the high dimensional space.

```
embedding['copyright']
```

```
array([ 0.28292 , -1.0518  ,  0.17169 , -0.19509 ,  1.1078  ,  0.35123 ,
       -1.1153  , -0.7574  ,  0.43468 , -0.23271 ,  0.56626 , -0.32403 ,
        0.20586 , -0.7068  , -0.37967 ,  0.2811  ,  0.66518 ,  0.066218,
       -0.49656 , -0.44746 , -0.76732 , -0.69151 ,  1.0989  ,  0.68884 ,
        1.0099  , -0.099243,  0.86778 , -0.74706 , -0.53393 , -0.98153 ,
        0.33981 ,  0.87678 ,  0.55539 ,  0.25211 , -0.093136, -0.12268 ,
       -0.18995 , -0.5596  ,  0.022301, -0.65089 ,  0.15131 ,  0.69039 ,
       -0.25731 , -0.10418 ,  0.3187  , -0.92036 , -0.080776, -0.72957 ,
       -0.062274, -0.30242 ,  1.108   ,  0.45579 , -0.24216 , -0.17086 ,
        0.47928 , -0.38471 ,  0.55145 , -0.56831 ,  2.1103  , -0.40532 ,
       -0.35658 ,  0.30065 , -1.1101  , -0.40778 ,  0.65017 , -0.10852 ,
        0.97684 ,  0.30761 ,  0.043331, -0.54005 , -0.36475 ,  0.72474 ,
        0.18027 ,  0.41623 , -0.32136 , -0.942   , -0.87021 ,  0.009036,
       -1.6118  ,  0.085464,  0.46778 , -0.99888 ,  0.97935 ,  0.54841 ,
       -0.51551 ,  0.32104 , -0.57828 , -0.43939 ,  0.20083 ,  0.48005 ,
       -0.094059, -0.27191 ,  0.04519 , -0.8264  ,  0.86619 ,  0.76831 ,
       -0.006007, -0.12633 ,  0.25886 , -0.37395 ], dtype=float32)
```

You can experiment with embeddings here:

<https://colab.research.google.com/github/YData123/sds265-fa24/blob/main/demos/embeddings/embeddings.ipynb>

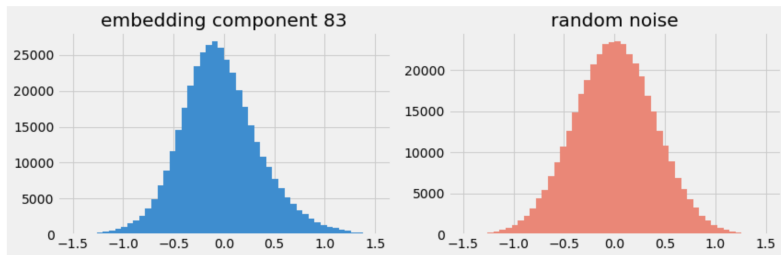
```
embedding.most_similar('copyright', topn=10)
```

```
[('infringement', 0.7312791347503662),  
 ('copyrights', 0.6773107051849365),  
 ('patent', 0.6438419818878174),  
 ('patents', 0.6030457019805908),  
 ('licensing', 0.602853536605835),  
 ('piracy', 0.6020364165306091),  
 ('copyrighted', 0.5973290205001831),  
 ('rights', 0.5897216200828552),  
 ('privacy', 0.5878084897994995),  
 ('ipr', 0.5850082635879517)]
```

You can experiment with embeddings here:

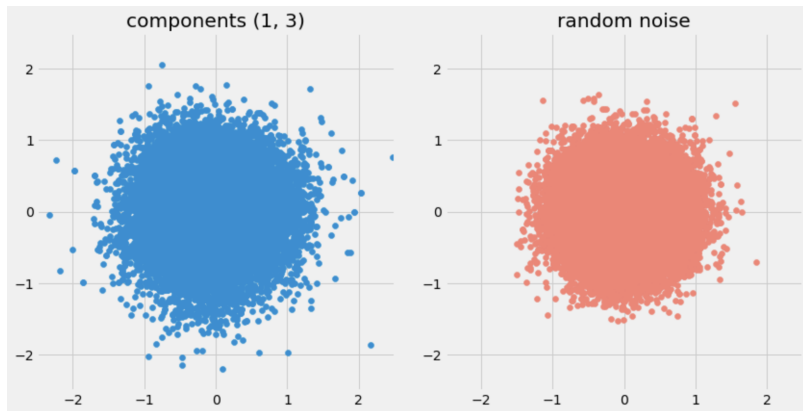
<https://colab.research.google.com/github/YData123/sds265-fa24/blob/main/demos/embeddings/embeddings.ipynb>

Embeddings look like random noise



But they encode a huge amount of useful information!

Embeddings look like random noise



But they encode a huge amount of useful information!

Analogies

Leads to vector representations of words with interesting properties.

For example, analogies:

`king is to man as ? is to woman`

Analogies

Leads to vector representations of words with interesting properties.

For example, analogies:

king is to man as ? is to woman

Paris is to France as ? is to Germany

Analogies

Leads to vector representations of words with interesting properties.

For example, analogies:

king is to man as ? is to woman

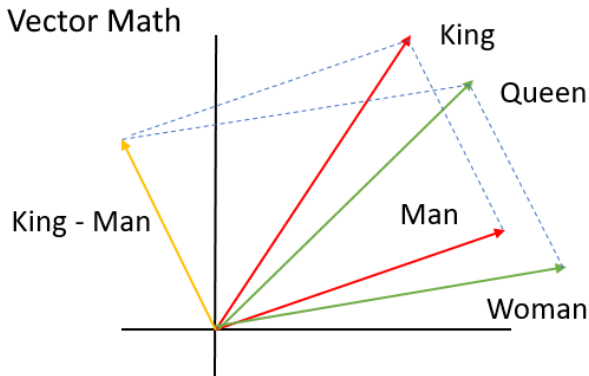
Paris is to France as ? is to Germany

$$\varphi(\text{king}) - \varphi(\text{man}) \stackrel{?}{\approx} \varphi(\text{queen}) - \varphi(\text{woman})$$

$$\hat{w} = \arg \min_w \|\varphi(\text{king}) - \varphi(\text{man}) + \varphi(\text{woman}) - \varphi(w)\|^2$$

Does $\hat{w} = \text{queen}$?

Word geometry



Word geometry

$\varphi(\text{king}) - \varphi(\text{man}) + \varphi(\text{woman})$:

queen
elizabeth
isabella
empress
throne
princess

Learned Analogies

Table 8: *Examples of the word pair relationships, using the best word vectors from Table 4 (Skip-gram model trained on 783M words with 300 dimensionality).*

Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Miami - Florida	Baltimore: Maryland	Dallas: Texas	Kona: Hawaii
Einstein - scientist	Messi: midfielder	Mozart: violinist	Picasso: painter
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev	Obama: Barack
Microsoft - Windows	Google: Android	IBM: Linux	Apple: iPhone
Microsoft - Ballmer	Google: Yahoo	IBM: McNealy	Apple: Jobs
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

Embeddings encode societal bias

$$\varphi(\text{worker}) - \varphi(\text{man}) + \varphi(\text{woman}):$$

nurse
migrant
immigrant
child
nurses
pregnant
teacher

$$\varphi(\text{worker}) - \varphi(\text{woman}) + \varphi(\text{man}):$$

working
factory
farmer
mechanic
laborer

Embeddings encode societal bias

$$\varphi(\text{scientist}) - \varphi(\text{man}) + \varphi(\text{woman}):$$

anthropologist
sociologist
psychologist
geneticist
biochemist

$$\varphi(\text{scientist}) - \varphi(\text{woman}) + \varphi(\text{man}):$$

geologist
engineer
astronomer
mathematician
science

Embeddings encode societal bias

$\varphi(\text{smart}) - \varphi(\text{girl}) + \varphi(\text{boy})$:

wise
better
guy
kind
good
kid

$\varphi(\text{smart}) - \varphi(\text{boy}) + \varphi(\text{girl})$:

sexy
pretty
incredibly
cute
exciting
funny

Summary: Word embeddings

- Word embeddings are vector representations of words, learned from cooccurrence statistics
- The models can be built using simple language modeling
- Surprising semantic relations are encoded in linear relations
- Embeddings are the “ground floor” representations in LLMs